

Publication Log – Full Testing Report

Project: Publication Log Web Application

Date: 22 Aug 2025

Owner: Basant Awad

SUT (System Under Test): Django-based web app for publication management (Auth, CRUD, Export)

1. Scope & Objectives

1.1 Scope

This report defines the full testing approach and evidence for the Publication Log system. It covers unit, integration, and system testing for Authentication, Publication CRUD, Search/Filter, and Export features.

1.2 Objectives

- Verify functional and non-functional requirements.
- Detect defects early via layered testing.
- Provide traceability from requirements → test cases → results.
- Produce repeatable steps and data.

1.3 Out of Scope

- Performance/load testing beyond basic responsiveness checks.
 - Cross-browser/UI pixel-perfection testing.
 - Localization/i18n beyond English.
-

2. Test Strategy

2.1 Levels of Testing

- **Unit Testing:** Validate models, utilities, and isolated view logic.
- **Integration Testing:** Validate interactions across models, views, URLs, forms, and auth.
- **System Testing (End-to-End):** Validate full user journeys: sign up/login → CRUD → export.

2.2 Test Types

- **Functional:** Authentication, role-based access, CRUD, export.
- **Negative:** Invalid inputs, unauthenticated access, permission checks.
- **Security (basic):** Auth required for restricted routes, CSRF respected via Django client.
- **Regression:** Smoke suite to run on each commit/PR.

2.3 Test Environment

- **Backend:** Python 3.x, Django 3.x/4.x
- **DB:** SQLite (test DB auto-created by Django)
- **OS:** Any (Windows/Linux/macOS)

- **Command:** `python manage.py test -v 2 -s` (use `-s` to show prints)

2.4 Entry & Exit Criteria

- **Entry:** App installs, migrations run, essential URLs configured.
 - **Exit:** All High/Critical severity defects closed or deferred with approval; system tests pass.
-

3. Requirements

3.1 Functional Requirements (FR)

- **FR-1 Auth:** Users can register, log in, and log out.
- **FR-2 Roles:** Role-based access (admin vs standard user) for restricted operations.
- **FR-3 Create:** Users can create a publication record.
- **FR-4 Read/List:** Users can view a list of publications.
- **FR-5 Update:** Users can edit a publication record.
- **FR-6 Delete:** Users can delete a publication record.
- **FR-7 Search/Filter:** Users can filter or search publications.
- **FR-8 Export CSV:** Users can export publications as CSV.
- **FR-9 Export PDF:** Users can export publications as PDF (if implemented; otherwise N/A).

3.2 Non-Functional Requirements (NFR)

- **NFR-1 Performance:** Typical CRUD actions respond within 1s on dev hardware.
- **NFR-2 Security:** Auth required for restricted endpoints; server-side validation present.
- **NFR-3 Usability:** Basic navigation to key features (Dashboard/List/Create/Edit/Delete/Export).
- **NFR-4 Reliability:** Tests run consistently with isolated test DB.

3.3 Test Requirements (TR)

- **TR-1** Ability to create users in tests and authenticate via Django client.
 - **TR-2** Publication model accessible (named `Publication`) or equivalent accessible via app registry.
 - **TR-3** Named URLs for CRUD and export exist (or tests adapt via discovery).
 - **TR-4** Forms/views accept minimal valid payload (e.g., title and author).
 - **TR-5** Export endpoints return correct content type (`text/csv` or `application/pdf`).
-

4. Test Data

- **User (standard):** username= `testuser`, password= `Pass12345!`
- **User (admin):** username= `adminuser`, password= `AdminPass123!`
- **Publication payload (baseline):** title= `"Deep Learning for X"`, author= `"Jane Doe"`, year= `2024` (if required), journal= `"AI Journal"` (optional)

Notes: Actual required fields may differ; the automated tests attempt to discover field names (`title`, `author(s)`, `year`) where possible.

5. Test Cases

Legend

- **Preconditions:** State before executing steps.
- **Expected:** Expected outcome for the system under test.
- **Actual:** To be filled after execution.
- **Status:** Pass/Fail (to be filled).

5.1 Unit Tests

ID	Title	Preconditions	Steps	Expected	Actual	Status
UT-01	Publication <code>__str__</code> returns readable title	Publication model exists	1) Instantiate Publication with title. 2) Cast to str.	String equals title or includes title.		
UT-02	Model create minimum valid record	Publication model & required fields known	1) Create Publication with minimal fields. 2) Save.	Record saved without error.		
UT-03	Auth: create & authenticate user	Django auth available	1) Create user. 2) <code>client.login()</code> .	Login success (True).		

5.2 Integration Tests (Views + URLs + Models)

ID	Title	Preconditions	Steps	Expected	Actual	Status
IT-01	Create Publication (POST)	Logged-in user	1) POST to create URL with payload. 2) Follow redirect.	HTTP 302 → list/ detail; record exists.		
IT-02	Edit Publication (POST)	Pub exists, logged-in	1) POST update with new title.	HTTP 302; DB shows updated title.		
IT-03	Delete Publication (POST)	Pub exists, logged-in	1) POST delete.	HTTP 302; record not found after.		
IT-04	List Publications (GET)	At least one pub	1) GET list.	HTTP 200; response contains known title.		

ID	Title	Preconditions	Steps	Expected	Actual	Status
IT-05	Search/Filter (GET)	Multiple pubs exist	1) GET list with <code>q</code> or <code>search</code> param.	HTTP 200; filtered result contains query term.		
IT-06	Export CSV (GET)	Pubs exist	1) GET export CSV.	HTTP 200; <code>text/csv</code> content type; non-empty body.		
IT-07	Export PDF (GET)	Pubs exist, feature enabled	1) GET export PDF.	HTTP 200; <code>application/pdf</code> content type; non-empty body.		
IT-08	Permissions: anon redirected	None	1) GET create/edit/delete as anon.	HTTP 302 to login.		

5.3 System Tests (End-to-End)

ID	Title	Preconditions	Steps	Expected	Actual	Status
ST-01	Full flow: Sign up → Login → Create → List → Export	App running	1) Sign up new user via signup URL or fallback create user. 2) Login. 3) Create pub. 4) List pubs. 5) Export CSV.	All steps succeed; export content type valid; created title visible in list.		
ST-02	Negative flow: Invalid form submit	Logged-in	1) POST create with missing required fields.	Form returns 200 with errors; record not created.		
ST-03	Role-based restriction	Admin & user exist	1) Try admin-only action as user. 2) Repeat as admin.	User denied (302/403); admin allowed.		

6. Traceability Matrix

Requirement	Test IDs
FR-1 Auth	UT-03, ST-01
FR-2 Roles	ST-03
FR-3 Create	IT-01, ST-01, ST-02

Requirement	Test IDs
FR-4 Read/List	IT-04, ST-01
FR-5 Update	IT-02
FR-6 Delete	IT-03
FR-7 Search/Filter	IT-05
FR-8 Export CSV	IT-06, ST-01
FR-9 Export PDF	IT-07
NFR-1 Performance	(Observed during IT & ST; add timing if measured)
NFR-2 Security	IT-08, ST-03
NFR-3 Usability	ST-01 (navigability)
NFR-4 Reliability	Full suite stability over runs

7. Execution & Results (to be filled after running tests)

- **Environment:** Python x.x, Django x.x, OS, Branch/Commit.
- **Command:** `python manage.py test -v 2 -s`

7.1 Summary

- Total Tests: __
- Passed: __
- Failed: __
- Skipped: __

7.2 Detailed Results

Fill the Actual/Status columns in Section 5 tables with runtime outputs. Attach console logs if needed (especially printed summaries from tests).

8. Risks, Assumptions, & Mitigations

- **Unknown field names/URL patterns:** Tests include discovery and graceful skips with clear messages. *Mitigation:* adjust the config block (field/URL names) in `tests.py` if discovery fails.
- **PDF export optional:** Mark as N/A if not implemented.
- **Role model differences:** If roles aren't implemented, restrict ST-03 accordingly.

9. Conclusions

If the suite passes with expected coverage, the Publication Log system meets its stated functional requirements (FR-1...FR-8/9). Failures should be triaged, fixed, and re-tested. Maintain this report per release with updated results, and run the test suite in CI for every change.

Appendix A – How to Run

```
# From project root
python manage.py test -v 2 -s

# Run only the app tests (replace `yourapp` name if needed)
python manage.py test yourapp -v 2 -s
```

Appendix B – Evidence Capture

- Save console output to a file:

```
python manage.py test -v 2 -s | tee test_run_YYYYMMDD.txt
```

- Attach CSV/PDF export files (if saved by tests) for validation.